

UNIVERSIDADE FEDERAL DO CEARÁ
CAMPUS CRATEÚS
CURSO DE CIÊNCIA DA COMPUTAÇÃO
DISCIPLINA: ALGORITMOS E GRAFOS

RELATÓRIO TÉCNICO
PROBLEMA DO CAIXEIRO VIAJANTE

Eric Albuquerque
Álvaro Freire

https://github.com/Erc-prog-xD/Grafos_trabfinal

Crateús – CE

2026

Sumário

1	Introdução	2
2	Método Proposto	2
2.1	Leitura da instância	2
2.2	Cálculo das distâncias	2
2.3	Árvore Geradora Mínima com Prim	3
2.4	Identificação dos vértices ímpares	3
2.5	Emparelhamento guloso	3
2.6	Construção do multigrafo	3
2.7	Ciclo euleriano	3
2.8	Conversão para ciclo hamiltoniano	4
2.9	Melhoria com 2-opt	4
2.10	Resumo do algoritmo	4
3	Implementação	5
4	Resultados e Discussões	6
4.1	Resultados obtidos	6
4.2	Análise dos resultados	6
4.3	Pontos fortes	6
4.4	Limitações	7
4.5	Possíveis melhorias	7
5	Conclusão	7
6	Uso de ferramentas de IA	7
6.1	Ferramentas utilizadas	8
6.2	Finalidades de uso	8
6.3	Prompts utilizados	8
7	Figuras do tsp51.tsp	9
8	Referências	11

1 Introdução

O Problema do Caixeiro Viajante, conhecido como TSP, consiste em encontrar um ciclo que visite todas as cidades exatamente uma vez e retorne à cidade inicial, buscando o menor custo total possível. Em termos de grafos, o problema procura um ciclo hamiltoniano de custo mínimo em um grafo ponderado.

Esse problema é importante porque aparece em diferentes situações práticas, como roteirização de entregas, planejamento de caminhos, logística, transporte e otimização de percursos. Além disso, o TSP é um problema computacionalmente difícil, o que torna comum o uso de métodos heurísticos ou aproximativos para encontrar boas soluções em tempo viável.

O objetivo deste trabalho é implementar e avaliar um método para resolver instâncias do TSP no formato TSPLIB, considerando cidades representadas por coordenadas em duas dimensões. O método desenvolvido busca construir um tour válido, comparar o custo obtido com uma solução ótima conhecida quando disponível e analisar o comportamento do algoritmo.

2 Método Proposto

O método implementado foi baseado na ideia geral do algoritmo de Christofides, com algumas adaptações para simplificar a implementação. A solução foi construída a partir de uma árvore geradora mínima, seguida pelo emparelhamento dos vértices de grau ímpar, construção de um multigrafo com algoritmo guloso, obtenção de um ciclo euleriano, transformação em ciclo hamiltoniano e melhoria final com a heurística 2-opt.

2.1 Leitura da instância

O programa realiza a leitura de arquivos no formato TSPLIB [1], considerando instâncias do tipo TSP com coordenadas bidimensionais. A leitura é feita a partir da seção `NODE_COORD_SECTION`, onde cada vertice possui um identificador e suas respectivas coordenadas x e y .

2.2 Cálculo das distâncias

A partir das coordenadas dos vertices, foi construída uma matriz de distâncias. Cada posição da matriz representa a distância entre duas cidades.

A distância utilizada foi a distância euclidiana arredondada, conforme o padrão `EUC_2D` da TSPLIB:

$$d_{ij} = \left\lceil 0,5 + \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \right\rceil \quad (1)$$

Essa matriz é usada em todas as etapas seguintes para calcular custos de arestas e o custo total do tour.

2.3 Árvore Geradora Mínima com Prim

Depois da criação da matriz de distâncias, o algoritmo constrói uma árvore geradora mínima usando o algoritmo de Prim adaptado de [2]. Essa árvore conecta todas as cidades com um custo inicial reduzido, sem formar ciclos.

No notebook, essa etapa é feita pela função `prim_matriz`, que percorre a matriz de distâncias e adiciona, a cada passo, a menor aresta que conecta um vértice já selecionado a um vértice ainda não selecionado.

2.4 Identificação dos vértices ímpares

Após a construção da árvore geradora mínima, são identificados os vértices que possuem grau ímpar. Para isso, o algoritmo percorre as arestas da árvore e contabiliza o grau de cada cidade.

Essa etapa é necessária porque, para construir um ciclo euleriano, todos os vértices do multigrafo devem possuir grau par.

2.5 Emparelhamento guloso

Os vértices de grau ímpar são emparelhados por meio de uma estratégia gulosa. O algoritmo seleciona um vértice ímpar e procura, entre os demais vértices ímpares ainda disponíveis, aquele com menor distância.

Essa abordagem não garante o emparelhamento perfeito mínimo global, mas possui implementação mais simples e menor custo computacional. Dessa forma, ela foi utilizada como uma adaptação viável para o trabalho.

2.6 Construção do multigrafo

O multigrafo é construído a partir da união das arestas da árvore geradora mínima com as arestas obtidas no emparelhamento dos vértices ímpares. Com isso, os graus dos vértices tendem a se tornar pares, permitindo a construção de um ciclo euleriano.

2.7 Ciclo euleriano

Para encontrar o ciclo euleriano, foi utilizada uma abordagem baseada no algoritmo de Hierholzer. Inicialmente, os vizinhos de cada vértice são ordenados em ordem decrescente usando `reverse=True`. Essa ordenação foi adotada porque o algoritmo remove os vizinhos com a operação `pop()`, que retira o último elemento da lista. Assim, ao

ordenar os vizinhos de forma decrescente, o último elemento passa a ser o menor vizinho disponível, permitindo um controle maior sobre a ordem de visitação.

Durante a execução, uma pilha armazena o caminho atual. Enquanto houver arestas disponíveis a partir do vértice no topo da pilha, uma dessas arestas é removida do grafo e o vértice vizinho é adicionado à pilha. Quando um vértice não possui mais arestas disponíveis, ele é removido da pilha e inserido na lista do ciclo.

Ao final, a lista obtida é invertida com `ciclo.reverse()`, pois os vértices são adicionados ao ciclo no momento de retorno da pilha, ficando em ordem inversa ao percurso euleriano real. Dessa forma, a inversão final permite retornar o ciclo euleriano na ordem correta, porém isso não influencia diretamente no peso do ciclo euleriano, mas na etapa posterior isso muda já que para a transformação em ciclo hamiltoniano, o caminho euleriano pego pode possuir um caminho com mais atalhos do que quando não utilizamos o `ciclo.reverse()`.

2.8 Conversão para ciclo hamiltoniano

Como a solução final do TSP deve visitar cada cidade exatamente uma vez, o ciclo euleriano é convertido em um ciclo hamiltoniano. Para isso, o algoritmo percorre o ciclo euleriano e mantém apenas a primeira ocorrência de cada cidade, ignorando as repetições. Ao final, a cidade inicial é adicionada novamente para fechar o ciclo.

2.9 Melhoria com 2-opt

Após obter o ciclo hamiltoniano inicial, foi aplicada a heurística 2-opt adaptada de [3]. Essa técnica tenta melhorar o tour trocando a ordem de partes do caminho. A ideia é inverter trechos do ciclo sempre que essa alteração reduzir o custo total. O processo continua enquanto forem encontradas melhorias.

Essa etapa foi importante porque o ciclo hamiltoniano gerado a partir do ciclo euleriano nem sempre apresenta uma boa ordem de visitação. Assim, o 2-opt ajuda a reduzir o custo final da solução.

2.10 Resumo do algoritmo

De forma resumida, o método executa as seguintes etapas:

1. Ler a instância no formato TSPLIB;
2. Criar a matriz de distâncias;
3. Gerar a árvore geradora mínima com Prim;
4. Encontrar os vértices de grau ímpar;

5. Emparelhar os vértices ímpares usando estratégia gulosa;
6. Unir as arestas da árvore com as arestas do emparelhamento;
7. Construir o multigrafo;
8. Encontrar o ciclo euleriano;
9. Remover repetições para formar o ciclo hamiltoniano;
10. Aplicar a heurística 2-opt;
11. Salvar o tour final no formato exigido.

3 Implementação

O algoritmo foi implementado em Python. Foram utilizadas bibliotecas auxiliares para operações matemáticas, manipulação de dados e geração de gráficos, como `math`, `matplotlib` e `pandas`.

A implementação foi organizada em funções separadas, facilitando a leitura e a manutenção do código. As principais funções implementadas foram:

- `ler_tsp_arquivo`: realiza a leitura da instância;
- `criar_matriz_distancias`: constrói a matriz de distâncias;
- `prim_matriz`: gera a árvore geradora mínima;
- `encontrar_vertices_impares`: identifica os vértices de grau ímpar;
- `emparelhamento_guloso`: realiza o emparelhamento dos vértices ímpares;
- `construir_multigrafo`: constrói o multigrafo;
- `ciclo_euleriano`: gera o ciclo euleriano;
- `ciclo_hamiltoniano`: remove repetições e gera o ciclo hamiltoniano;
- `dois_opt`: melhora o tour por busca local;
- `salvar_tour`: salva a solução final no formato `.tour`.

Além da geração da solução, o notebook também possui funções de visualização gráfica, utilizadas para exibir o grafo completo, a árvore geradora mínima, o multigrafo, o ciclo hamiltoniano inicial, o ciclo hamiltoniano melhorado e o tour ótimo, quando disponível.

4 Resultados e Discussões

Os testes foram realizados utilizando a instância `tsp51.tsp` além de instâncias maiores para checar a agilidade da execução. A execução do notebook permitiu visualizar todas as etapas principais do algoritmo, desde a construção do grafo completo até o tour final melhorado com 2-opt.

4.1 Resultados obtidos

Durante a execução, foram obtidos os seguintes valores:

Tabela 1: Resultados obtidos na instância testada		
Métrica	Valor	Figura
Instância utilizada	tsp51.tsp	Figura 1
Custo da árvore geradora mínima	375	Figura 2
Custo do multigrafo	213	Figura 3
Custo do hamiltoneano	553	Figura 4
Custo final após 2-opt	438	Figura 5
Custo ótimo	426	Figura 6
Razão em relação ao ótimo	1,028	–

4.2 Análise dos resultados

A árvore geradora mínima apresentou um custo inicial baixo, pois seu objetivo é conectar todos os vértices com o menor custo possível, sem formar ciclos. Entretanto, ela não representa uma solução válida para o TSP, já que não forma um ciclo hamiltoniano.

Após o emparelhamento dos vértices ímpares e a construção do multigrafo, foi possível obter um ciclo euleriano. Em seguida, a remoção dos vértices repetidos gerou um ciclo hamiltoniano válido.

A aplicação do 2-opt melhorou a solução final, pois reduziu cruzamentos e reorganizou partes do caminho. Com isso, o custo final do tour foi reduzido em relação ao ciclo hamiltoniano inicial.

4.3 Pontos fortes

O método possui como ponto forte a simplicidade de implementação. A divisão em etapas facilita o entendimento do algoritmo e permite visualizar claramente como a solução é construída.

Outro ponto positivo é o uso do 2-opt, que melhora a qualidade da solução sem exigir uma implementação muito complexa.

4.4 Limitações

A principal limitação do método está no emparelhamento guloso. Como ele sempre escolhe a menor opção local para cada vértice, pode não encontrar o melhor conjunto global de pares.

Além disso, a etapa de conversão do ciclo euleriano para hamiltoniano depende da ordem em que os vértices aparecem no ciclo. Isso pode gerar uma solução inicial que ainda precisa ser bastante melhorada pelo 2-opt.

Outra limitação é que o método não garante encontrar a solução ótima, pois se trata de uma abordagem heurística.

4.5 Possíveis melhorias

Como melhoria futura, seria possível substituir o emparelhamento guloso por um algoritmo de emparelhamento perfeito mínimo mais preciso. Também seria possível testar diferentes vértices iniciais no ciclo euleriano ou aplicar outras heurísticas de melhoria, como 3-opt.

Outra possibilidade seria executar o 2-opt a partir de diferentes soluções iniciais e escolher o melhor resultado obtido.

5 Conclusão

Este trabalho apresentou a implementação de um método heurístico para o Problema do Caixeiro Viajante. A abordagem utilizou uma adaptação baseada no algoritmo de Christofides, combinando árvore geradora mínima, emparelhamento guloso, ciclo euleriano, ciclo hamiltoniano e melhoria local com 2-opt.

O algoritmo foi capaz de gerar um tour válido para a instância testada, visitando todas as cidades exatamente uma vez e retornando à cidade inicial. A aplicação do 2-opt contribuiu para melhorar a qualidade da solução, reduzindo o custo final do percurso.

Apesar de não garantir a solução ótima, o método apresentou uma estrutura clara, funcional e adequada para resolver instâncias do TSP de forma heurística. As principais limitações estão relacionadas ao uso do emparelhamento guloso e à dependência da ordem do ciclo euleriano gerado.

6 Uso de ferramentas de IA

Durante o desenvolvimento deste trabalho, ferramentas de inteligência artificial foram utilizadas como apoio para organização da estrutura do relatório, revisão textual e esclarecimento de conceitos relacionados ao Problema do Caixeiro Viajante.

A implementação, os testes, a interpretação dos resultados e as decisões técnicas foram realizados pelos integrantes da equipe.

6.1 Ferramentas utilizadas

- ChatGPT.

6.2 Finalidades de uso

- Apoio na estruturação do relatório;
- Apoio na revisão textual;
- Apoio na explicação de conceitos;
- Apoio na formatação em LaTeX.
- Apoio na criação de gráficos usando a biblioteca matplotlib

6.3 Prompts utilizados

- Criação de estrutura em LaTeX para relatório técnico sobre o Problema do Caixeiro Viajante;
- Organização dos tópicos do relatório com base no notebook implementado;
- Revisão da descrição do método proposto.
- Criação de graficos utilizando a matriz, coordenadas e caminhos atuais.

7 Figuras do tsp51.tsp

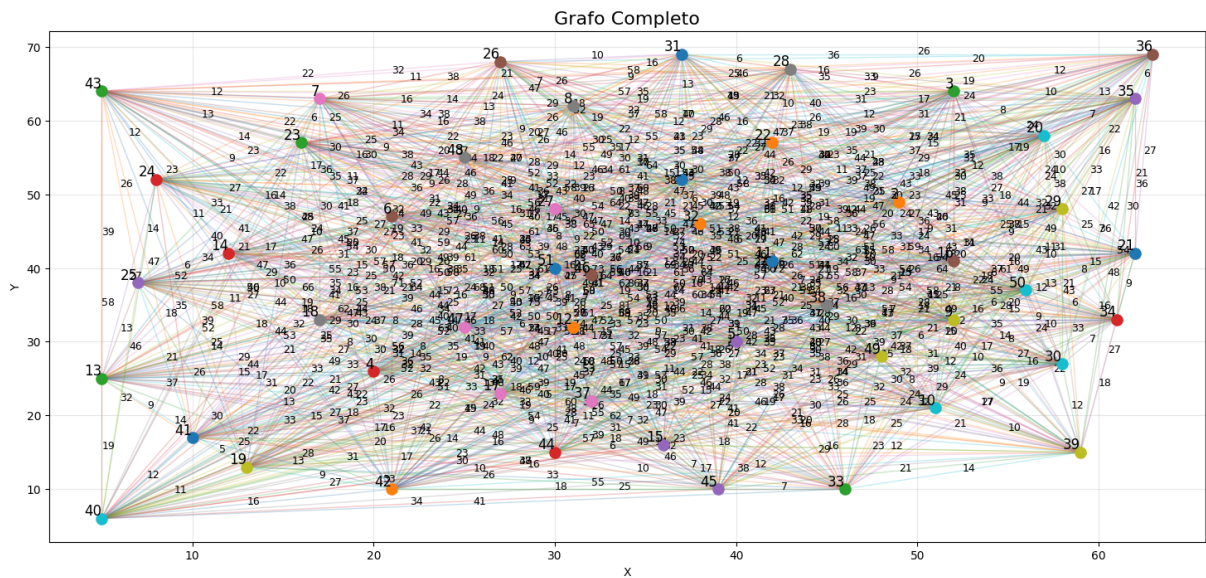


Figura 1: Grafo completo inicial.

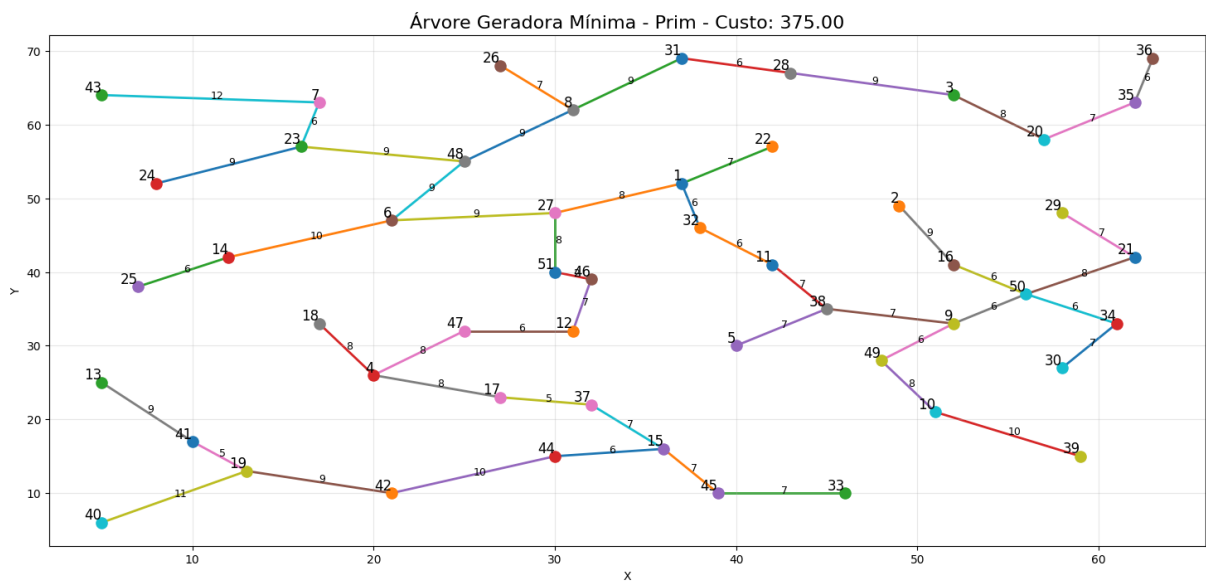


Figura 2: Grafo árvore geradora mínima.

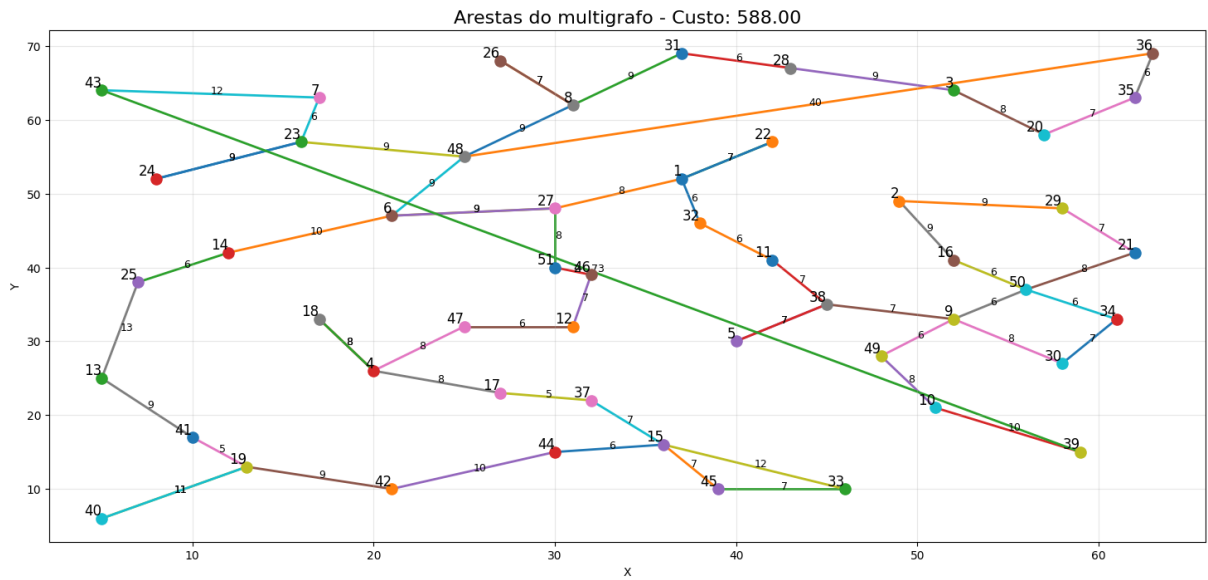


Figura 3: Grafo custo multigrafo.

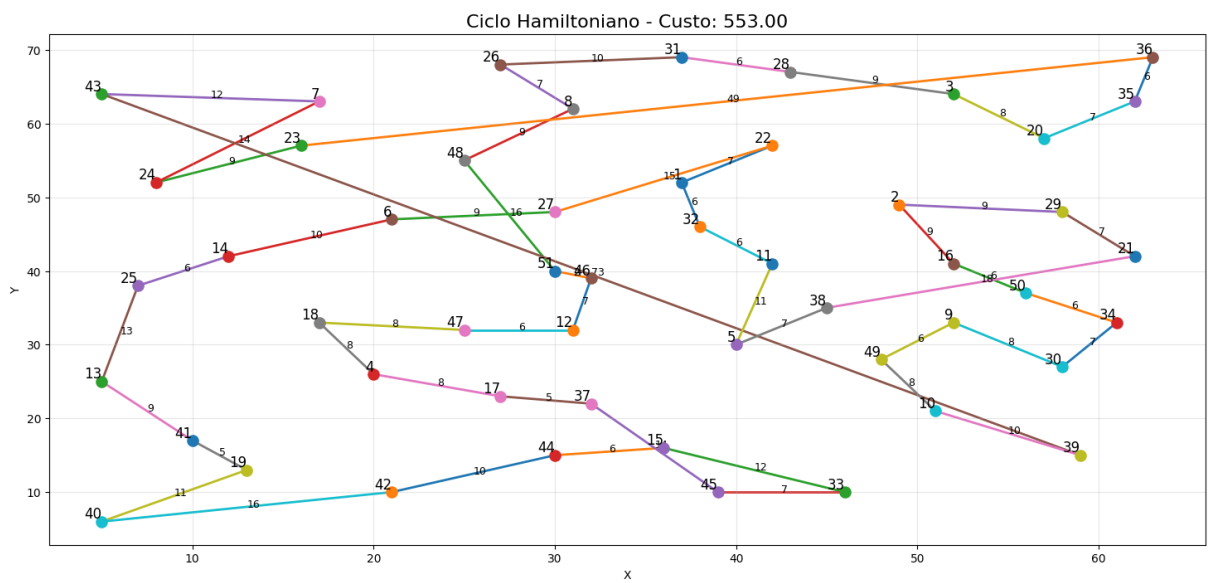


Figura 4: Grafo custo ciclo hamiltoniano.

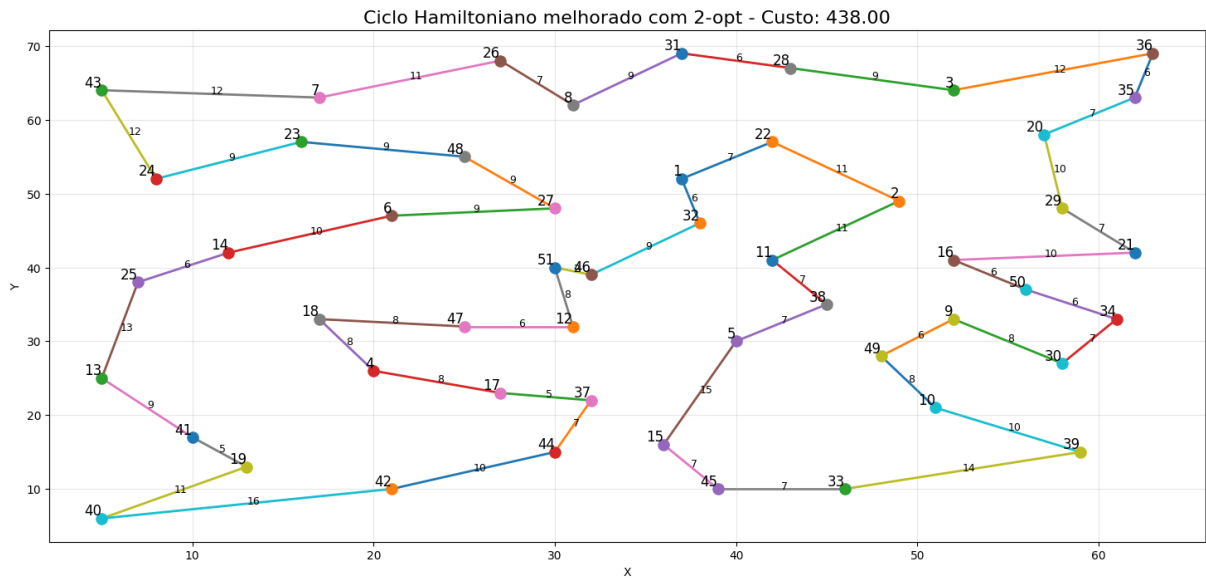


Figura 5: Grafo tour final melhorado com 2-opt.

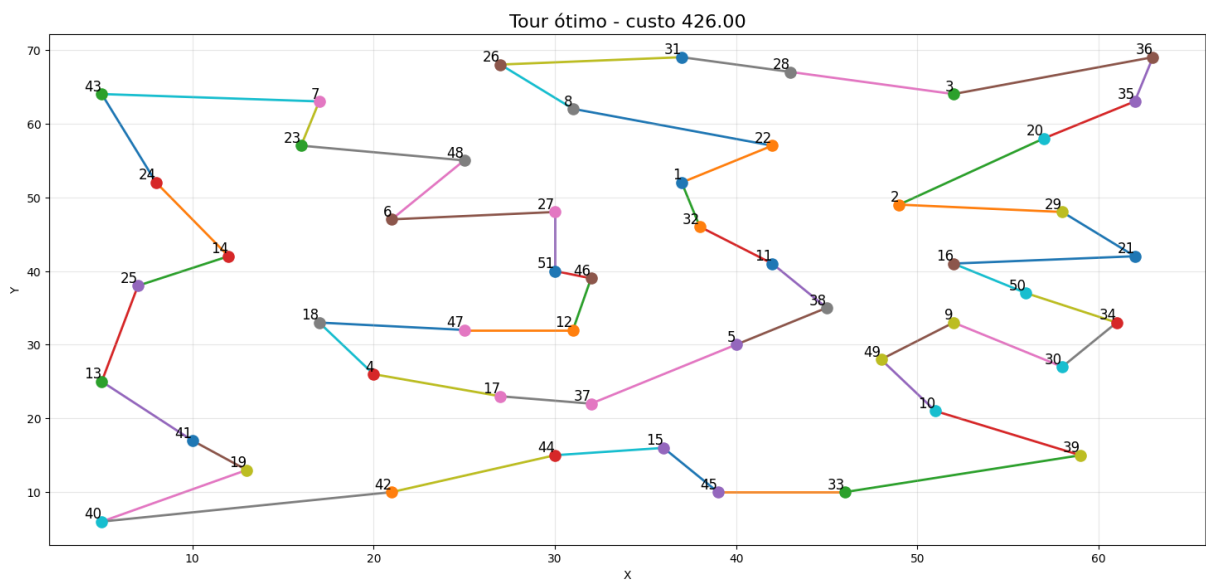


Figura 6: Grafo custo ótimo.

8 Referências

Referências

- [1] TSPLIB. *Travelling Salesman Problem Library*. Disponível em: <http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/>. Acesso em: 02 jun. 2026.
- [2] PROGRAMIZ. *Prim's Algorithm*. Disponível em: <https://www.programiz.com/dsa/prim-algorithm>. Acesso em: 02 jun. 2026.

- [3] SLOW AND STEADY BRAIN. *Traveling Salesman Problem*. Disponível em: <https://slowandsteadybrain.medium.com/traveling-salesman-problem-ce78187cf1f3>. Acesso em: 02 jun. 2026.